

Swinburne University of Technology Sarawak Campus
Faculty of Engineering, Computing and Science

LABORATORY COVER SHEET

Subject Code: COS30008
Subject Title: Data Structures and Patterns
Lab Number and Title: 4, Introduction to Iterator Design Pattern
Lecturer: Dr. Mark Tee & Isaac Asante

Summary

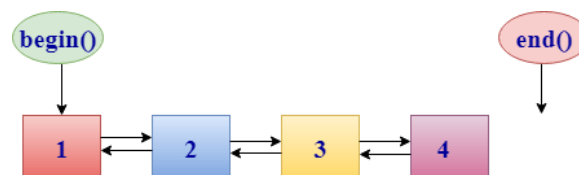
In this 4th lab session, you must quickly get up to speed with the following:

- a) Implementing a simple Input Iterator.
- b) Learn how it can quickly be applied to different Containers.
- c) How to modify it into a Forward Iterator (capable of both Reading and Writing).

Iterators

This is a design pattern that aims to provide a standard interface (common operators like ++, --, * and ==) for programs to use different Containers (data structures like arrays, structs, linked-list and etc.). This tutorial example applies the design pattern on Arrays, but later on, you can build Iterators to work with other Containers.

This way, even if you change the type of Container used, the program remains lightly changed because it accesses Container content using the same operators (++ , --, * and ==) instead.



Part A: Building a Simple Input Iterator (Read Forward only)

1. Start a new C++ Console Program and dimension a string array on the stack, in Main. The goal is to make the following For..Loop working so we can print the array contents.

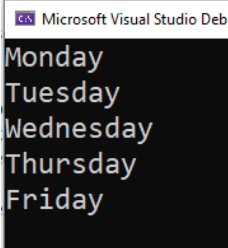
```
#include <iostream>
#include "ArrayIterator.h"

using namespace std;

int main()
{
    // Step 1: create a sample Array to act as our Container.
    const int sArrSize = 5;
    string sArr[sArrSize] = { "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" };

    /*
     * Step 2: Quick example of how all the Forward Input Iterator functions are used.
     * The life of the "ArrayIterator" object only lasts within the scope of the For Loop.
     * Refer to now the ArrayIterator class is developed.
     */
    for (ArrayIterator iter(sArr, sArrSize); iter != iter.end(); iter++)
    {
        cout << *iter << endl;
    }

    return 0;
}
```



```
Monday
Tuesday
Wednesday
Thursday
Friday
```

2. Create the ArrayIterator class (new .h and .cpp files). Define your header file like so:

```
#pragma once
#include <iostream>
using namespace std;

class ArrayIterator
{
private:
    // The attributes here are meant to reference (not copy) a container.
    const string * fArrayElements;
    const int fLength;

    // We use this index to keep track of which element is currently selected.
    int fIndex;

public:
    // Constructor: initialize the array pointer, size and starting element index.
    // "int aStart = 0" is interesting because this helps assign a value by default.
    ArrayIterator(const string aArray[], const int aLength, int aStart = 0);

    // Input Iterator Operators
    const string& operator*() const; //For accessing selected element's value
    ArrayIterator& operator++(); //Step Forward - return new position
    ArrayIterator operator++(int); //Step Forward - return old position
    bool operator==(const ArrayIterator& aOther) const; //True if array and index match
    bool operator!=(const ArrayIterator& aOther) const; //Opposite of the above

    ArrayIterator begin() const; //Return a copy of this iterator, index at beginning
    ArrayIterator end() const; //Return a copy of this iterator, index at the end
};
```

Find out:

- Looks familiar? Follow the lecture slide's example for the Integer Array Iterator to build this one's implementation (.cpp file).

3. Now add the following example code in the bottom of your Main Function, before you return 0.

```
int main()
{
    // Step 1: create a sample Array to act as our Container.
    const int sArrSize = 5;
    string sArr[sArrSize] = { "Monday", "Tuesday", "Wednesday", "Thursday", "Friday"};

    /*
     * Step 2: Quick example of how all the Forward Input Iterator functions are used.
     * The life of the "ArrayIterator" object only lasts within the scope of the For Loop.
     * Refer to now the ArrayIterator class is developed.
     */
    for (ArrayIterator iter(sArr, sArrSize); iter != iter.end(); iter++)
    {
        cout << *iter << endl;
    }

    // Step 3, try doing this with a Dynamic Array of Strings.
    int sDArrSize = 0;
    cout << endl << "Enter New Array Size:"; cin >> sDArrSize;

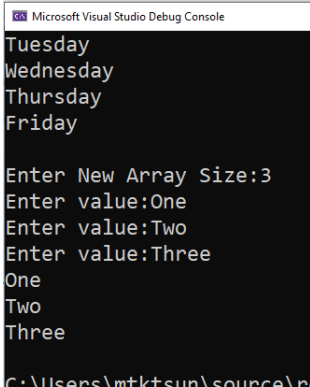
    string* sDArr = new string[sDArrSize];

    for (int a = 0; a < sDArrSize; a++)
    {
        cout << "Enter value:"; cin >> sDArr[a];
    }

    for (ArrayIterator iter(sDArr, sDArrSize); iter != iter.end(); iter++)
    {
        cout << *iter << endl;
    }

    delete[] sDArr; //Don't forget to delete the whole Dynamic Array

    return 0;
}
```



```
Microsoft Visual Studio Debug Console
Tuesday
Wednesday
Thursday
Friday
Enter New Array Size:3
Enter value:One
Enter value:Two
Enter value:Three
One
Two
Three
C:\Users\mtktsun\source\p
```

Find out:

- Without any modification, does the Iterator work for Dynamic Arrays? Why?

4. Time to modify this class so it becomes a Forward Iterator (able to Read & Write forwards). How will you do it? Hint: use the * operator to read and change values stored in the targeted element.

Follow-up Challenge:

- Add operator functions into the Iterator to turn it into a Bidirectional Iterator (Read & Write in Both Directions).

Part B: Challenge – Student Score Iterator Case Study

Examine the 5 source files (Main.cpp, Student.h, Student.cpp, ScoreIterator.h, ScoreIterator.cpp). This program is an example scenario where a custom class (Student) is designed to work with an Integer Array Iterator (ScoreIterator).

In the Main Function, notice that only the “Student.h” header file is included. Where is the Iterator’s header file?

Here, each “scores” array holds 6 scores/marks that are assigned per student (“John” and “Jane”), along with some values for their attributes.

Note how “auto” is a way to determine the right datatype for storing “students” via a new C++ technique called [Type Inference](#).

Notice how the Iterator is used, similar to Part A.

```
#include <iostream>
#include "Student.h"

int main()
{
    // Creating scores arrays, intended for 2 different students
    int scores1[] = { 90, 85, 88, 80, 82 };
    int scores2[] = { 78, 92, 87, 85, 89 };

    /*
     * Creating the students on the Stack.
     * Take note of how the arrays are passed to the constructors.
     */
    Student s1("John", 20, scores1, 5);
    Student s2("Jane", 21, scores2, 5);

    /*
     * Creating an array of custom data types (Student objects).
     */
    Student students[] = { s1, s2 };

    /*
     * Iterating through the students.
     * Do you see how we use the "auto" keyword in C++ to auto-detect the data types?
     * Also, note that the students are accessed by reference.
     */
    for (auto& student : students)
    {
        std::cout << "Name: " << student.getName()
                  << "\nAge: " << student.getAge()
                  << "\nScores: ";

        /*
         * Iterating through each student's scores using the forward iterator.
         * Note the overloaded behaviour of the !=, ++, and * operators.
         */
        for (ScoreIterator it = student.begin(); it != student.end(); ++it)
        {
            std::cout << *it << " ";
        }
        // Separate each student details with an empty line.
        std::cout << "\n" << std::endl;
    }
    return 0;
}
```

This is the “Student” class header (Student.h), built for representing each student profile which includes Name, Age, Number of Scores and the array of Scores.

```
#include <string>
#include "ScoreIterator.h" // Include the new header

class Student
{
private:
    std::string name;
    int age;
    int scores[5]; // assuming a fixed size of 5 for simplicity
    int numScores;

public:
    // Constructors (default and parameterized)
    Student();
    Student(const std::string& n, int a, const int s[], int size);

    // Member functions to access private data (getters)
    std::string getName() const;
    int getAge() const;
    int* getScores();
    int getNumScores() const;

    // Functions to get iterators for scores
    ScoreIterator begin();
    ScoreIterator end();
};
```

Notice:

- The Student Class header includes the ScoreIterator header. This is why Main.cpp can see the definitions of ScoreIterator.
- In this example, notice that begin() and end() implementations are done in the Student Class, not the ScoreIterator. Why?
- What is “numScores” attribute used for? See Student.cpp.

This is the implementation (in Student.cpp):

```
#include "Student.h"

// Default constructors (everything set to 0)
Student::Student() : name(""), age(0), numScores(0)
{
    for (int i = 0; i < 5; i++)
    {
        scores[i] = 0;
    }
}

// Parameterized constructor (accepting a list of scores)
Student::Student(const std::string& n, int a, const int s[], int size) : name(n), age(a), numScores(size)
{
    for (int i = 0; i < size; ++i)
    {
        scores[i] = s[i];
    }
}

// Getter methods
std::string Student::getName() const { return name; }

int Student::getAge() const { return age; }

int* Student::getScores() { return scores; }

int Student::getNumScores() const { return numScores; }

// Find the start of the containers.
ScoreIterator Student::begin() { return ScoreIterator(scores); }

// Find past-the-end of the container.
ScoreIterator Student::end() { return ScoreIterator(scores + numScores); }
```

Finally, take a look at the Iterator Definition (ScoreIterator.h):

```
class ScoreIterator
{
private:
    int* ptr;
public:
    ScoreIterator(int* p);

    // Dereference operator
    int& operator*();

    // Increment operators
    ScoreIterator& operator++(); // prefix ++
    ScoreIterator operator++(int); // postfix ++

    // Comparison operators
    bool operator==(const ScoreIterator& other) const;
    bool operator!=(const ScoreIterator& other) const;
};
```

Observe how it is implemented (in ScoreIterator.cpp):

```
#include "ScoreIterator.h"

ScoreIterator::ScoreIterator(int* p) : ptr(p) {}

/*
 * Overloading the behaviour of the operators.
 */

// Acting as deferencing
int& ScoreIterator::operator*()
{
    return *ptr;
}

// Prefix increment
ScoreIterator& ScoreIterator::operator++()
{
    ++ptr;
    return *this;
}

// Postfix increment
ScoreIterator ScoreIterator::operator++(int)
{
    ScoreIterator temp = *this;
    ++ptr;
    return temp;
}

// Equality
bool ScoreIterator::operator==(const ScoreIterator& other) const
{
    return ptr == other.ptr;
}

// Inequality
bool ScoreIterator::operator!=(const ScoreIterator& other) const
{
    return ptr != other.ptr;
}
```

Find Out:

- Is there an Index field?
- How does this Iterator keep track of which element it is currently targeting?
- How does == and != operators work, without begin() and end() inside this class?
- Which Type of Iterator is this?

```
Microsoft Visual Studio Debug Console
Name: John
Age: 20
Scores: 90 85 88 80 82

Name: Jane
Age: 21
Scores: 78 92 87 85 89

C:\Users\mtktsun\source\repos\COS30008_2023S2_Week4\x64\Debug\COS30008_2023S2_Week4.exe (process 17524) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

The challenge is to implement this program in your own Visual Studio, study how it operates and then answer interview questions during the sign-off to demonstrate your understanding.